

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **1**
Topic : **3**

Comparing ACID and BASE

Objectives

This session will give the knowledge about

- Understanding RDBMS and ACID transactions
- Understanding NoSQL and BASE transactions
- ACID vs BASE

Two methods of reliable database transactions

Transaction control is important in distributed computing environments with respect to performance and consistency.

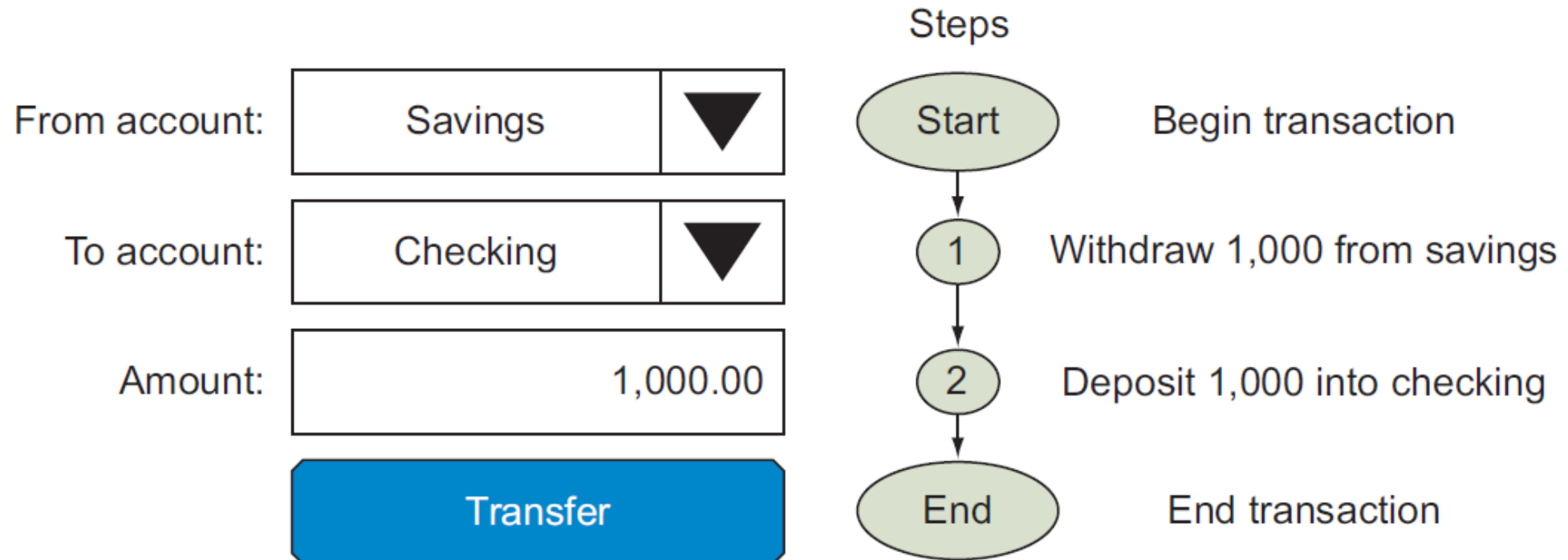
Typically one of **two types of transaction control models** are used:

- ACID, used in RDBMS, and
- BASE, found in many NoSQL systems.

The **difference between these models is in the amount of effort required by application developers and the location (tier) of the transactional controls.**

Database transactions - Example

Consider this example



Database transactions - Example

Two Operations: The funds are subtracted from your savings account and then added to your checking account.

Transaction management is the process of making sure that these two operations happen together as a single unit of work or not at all.

RDBMS have made it easy for programmers to make transactions reliable by wrapping critical transactions in statements that indicate where transactions begin and end.

These are often called **BEGIN TRANSACTION** and **END TRANSACTION** statements.

By adding them, developers can get high-reliability transaction support.

RDBMS transaction control using ACID

RDBMSs maintain transaction control by using atomic, consistent, independent, and durable (ACID) properties to insure transactions are reliable.

Atomicity - In the banking transaction example, we said that the exchange of funds from savings to checking must happen as an all-or-nothing transaction. The technical term for this is atomicity

Consistency - In the banking transaction example, we talked about the fact that when moving funds between two related accounts, the total account balance must never change. This is the principle of consistency

RDBMS transaction control using ACID

Isolation - Isolation refers to the concept that each part of a transaction occurs without knowledge of any other transaction. For example, the transaction that adds funds doesn't know about the transaction that subtracts funds from an account.

Durability - Durability refers to the fact that once all aspects of a transaction are complete, it's permanent. Once the transfer button is selected, you have the right to spend the money in your checking account.

Drawbacks of RDBMS

It's very complex and relational databases can be expensive.

Many RDBMSs restrict transaction location to a single CPU.

Although ACID transactions is complex, there are well-known and well publicized strategies to do this.

All of them depend on locking resources, putting extra copies of the resources aside, performing the transaction and then, if all is well, unlocking the resources.

ACID systems focus on the consistency and integrity

Non-RDBMS transaction control using BASE

Consider you have a website that relies on computers all over the world.

- A computer in USA manages your inventory,
- Product photos are on an image database in UK,
- Tax calculations are performed in India, and
- Your accounting system is in Australia.

What if one site goes down?

Should you tell your customers to check back in 20 minutes while you solve the problem?

Is it realistic to use ACID software for every order that comes in?

Non-RDBMS transaction control using BASE

- The BASE model (Basically Available, Soft state, Eventual consistency) is often used instead of the traditional ACID properties to handle transactions.
- BASE is designed to achieve high availability and scalability, sacrificing some consistency guarantees for better performance and fault tolerance.
- BASE guarantees that the system remains operational even when some of its components fail.
- By relaxing consistency requirements, BASE systems can provide faster responses to queries and updates, reducing the overall latency experienced by end-users.

Non-RDBMS transaction control using BASE

Basically Available:

- The system guarantees availability, meaning **it will always respond to requests, even if the responses are not immediately consistent.**
- This ensures that the system remains operational under various conditions, including network partitions.
- A retail website like **Amazon** needs to handle millions of transactions during a sale event. To ensure the system remains available, the database might allow some flexibility in data consistency. For instance, if a product's inventory is temporarily out of sync due to high demand, the website still allows users to add items to their cart, and updates the inventory asynchronously.

Non-RDBMS transaction control using BASE

Soft State:

- The state of the system may change over time, even without input, due to the asynchronous propagation of updates.
- **The system does not need to be immediately consistent**, allowing for greater flexibility in handling data.
- A distributed database used by a global social media platform like **Facebook** might have nodes spread across different regions. As users update their profiles or post new content, these changes might not be immediately reflected in all nodes. The system's state is considered "soft" because it is continually changing and eventually synchronizes across all nodes.

Non-RDBMS transaction control using BASE

Eventual Consistency:

- The system guarantees that, given enough time, **all replicas will converge to the same state**.
- This allows for temporary inconsistencies while ensuring long-term data integrity.
- In a NoSQL database like **Cassandra used by Netflix**, when a user updates their watchlist, the update might not be instantly visible across all devices due to network latency or partitioning. However, within a short period, all copies of the data across different nodes will eventually be consistent, ensuring that the user's watchlist is accurately reflected on all devices.

Conclusion

- Unlike RDBMSs that focus on consistency, BASE systems **focus on availability**
- **They relax the rules** and allow reports to run even if not all portions of the database are synchronized.
- BASE systems tend to be simpler and faster because **they don't have to write code that deals with locking and unlocking resources.**
- BASE systems are ideal for web storefronts, where filling a shopping cart and placing an order is the main priority.
- NoSQL systems are **highly decentralized** and ACID guarantees may not be necessary, so they use BASE and take a more relaxed approach

ACID vs BASE

ACID	BASE
Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventual consistency
Strong consistency; transactions always bring the database from one valid state to another	Eventual consistency; the system will become consistent over time
May sacrifice availability for consistency, especially during network partitions	Prioritizes availability, always responds to requests even if data is inconsistent
Maintains a hard state; transactions always leave the database in a valid state	Maintains a soft state; data may change over time until it becomes consistent

ACID vs BASE

ACID	BASE
Transactions are atomic, all-or-nothing	Transactions are not necessarily atomic; some parts may be partially complete
Transactions are isolated, intermediate states are not visible	Intermediate states may be visible, and updates propagate asynchronously
Once a transaction is committed, it remains so even in the event of a system crash	Durability is less emphasized, focusing more on availability and partition tolerance
Suitable for financial systems, banking, transaction processing where data integrity is crucial	Suitable for distributed systems, social media platforms, online shopping where high availability is critical
PostgreSQL, MySQL, Oracle	Cassandra, DynamoDB, Couchbase

At Conclusion

- ACID and BASE aren't rigid points on a line; they lie on a continuum where organizations and systems can decide where and how to architect systems.
- They may allow ACID transactions on some key areas but relax them in others.
- Some database systems offer both options by changing a configuration file or using a different API.
- When the amount of data you manage exceeds the size of your current system and you need to **use database sharding** to keep systems running and minimize downtime.

Summary

This session will give the knowledge about

- Understanding RDBMS and ACID transactions
- Understanding NoSQL and BASE transactions
- ACID vs BASE